Kyde Food — Documentação do Banco de Dados

Banco `alimentacao` · Autor: Kauã · Outubro de 2026

1. Visão geral

O banco de dados `alimentacao` guarda os dados do totem de autoatendimento **Kyde Food**: o cardápio, os clientes, os pedidos, os itens de cada pedido e os pagamentos.

Característica	Valor
Nome do banco	`alimentacao`
SGBD	MariaDB 10.4.32 (XAMPP), compatível com MySQL
Ferramenta de administração	phpMyAdmin 5.2.1
Mecanismo das tabelas	InnoDB
Conjunto de caracteres	`utf8mb4`, com ordenação `utf8mb4_general_ci`
Quantidade de tabelas	6 (`cadastro`, `comanda`, `produto`, `pedido`, `item_pedido`, `pagamento`)
Script de criação	`alimentacao.sql`
Script de dados iniciais	`produtos.sql`

2. Modelo lógico

2.1 Entidades e relacionamentos

- **comanda** representa o cliente que faz o pedido (nome e CPF ou CNPJ). Uma comanda pode ter **vários pedidos**.
- **pedido** representa uma compra feita no totem. Cada pedido pertence a **uma comanda** e tem **um pagamento**.
- **produto** é cada item do cardápio. Um produto pode aparecer em **vários itens de pedido**.
- **item_pedido** liga um pedido a um produto, com a quantidade e o valor da linha. Um pedido tem **vários itens**.
- **pagamento** registra a forma de pagamento, o valor e o tipo de retirada de um pedido.

- **cadastro** é uma tabela do projeto original que não é usada pela aplicação atual (veja a seção 6).

2.2 Relacionamentos (chaves estrangeiras)

Tabela filha	Coluna	Tabela mãe	Coluna	Cardinalidade	Restrição no banco
`pedido`	`ID_comanda`	`comanda`	`ID_comanda`	N para 1	`fk_pedido_comanda`
`pagamento`	`ID_pedido`	`pedido`	`ID_pedido`	1 para 1 (na prática)	`fk_pagamento_pedido`
`item_pedido`	`ID_produto`	`produto`	`ID_produto`	N para 1	`fk_produto`
`item_pedido`	`ID_pedido`	`pedido`	`ID_pedido`	N para 1	apenas índice `fk_pedido` (sem chave estrangeira criada)

2.3 Resumo do fluxo dos dados

```
comanda 1 ──── N pedido 1 ──── 1 pagamento
                    |
                    1
                    |
                    N
               item_pedido N ──── 1 produto
```

3. Dicionário de dados

Todas as colunas são `NOT NULL`. As chaves primárias são `AUTO_INCREMENT`.

3.1 Tabela `produto`

Cardápio da lanchonete, com nome, descrição e preço de cada item.

Coluna	Tipo	Chave	Descrição
`ID_produto`	int(11)	PK	Identificador do produto
`nome_produto`	varchar(45)		Nome do lanche
`descricao_produto`	varchar(150)		Descrição, com os ingredientes
`preco_produto`	decimal(10,2)		Preço unitário em reais

3.2 Tabela `comanda`

Cliente identificado durante o pedido.

Coluna	Tipo	Chave	Descrição
`ID_comanda`	int(11)	PK	Identificador da comanda
`nome_comanda`	varchar(45)		Nome informado no cadastro
`CPF_comanda`	varchar(14)		CPF (11 dígitos) ou CNPJ (14 dígitos), gravado somente com números

3.3 Tabela `pedido`

Cabeçalho de cada pedido feito no totem.

Coluna	Tipo	Chave	Descrição
`ID_pedido`	int(11)	PK	Número do pedido, mostrado ao cliente na confirmação
`ID_comanda`	int(11)	FK	Comanda (cliente) dona do pedido
`data_pedido`	varchar(14)		Data e hora no formato `AAAAMMDDHHMMSS` (por exemplo, `20261015183045`)
`status_pedido`	varchar(20)		Situação do pedido. A aplicação grava `Em preparo`
`valorTotal_pedido`	decimal(10,2)		Soma dos valores dos itens

3.4 Tabela `item_pedido`

Itens de cada pedido.

Coluna	Tipo	Chave	Descrição
`ID_item_pedido`	int(11)	PK	Identificador do item
`ID_pedido`	int(11)	índice	Pedido ao qual o item pertence
`ID_produto`	int(11)	FK	Produto escolhido
`quantidade_item_pedido`	int(11)		Quantidade comprada

Coluna	Tipo	Chave	Descrição
`valor_item_pedido`	decimal(10,2)		Valor da linha: preço do produto multiplicado pela quantidade

3.5 Tabela `pagamento`

Pagamento de cada pedido.

Coluna	Tipo	Chave	Descrição
`ID_pagamento`	int(11)	PK	Identificador do pagamento
`ID_pedido`	int(11)	FK	Pedido pago
`valorTotal_pagamento`	decimal(10,2)		Valor pago (igual ao total do pedido)
`formaPagamento`	varchar(20)		`PIX`, `Cartão de Crédito`, `Cartão de Débito` ou `Dinheiro`
`retiradaAlimento`	varchar(20)		`Comer aqui` ou `Para levar`

3.6 Tabela `cadastro`

Tabela do projeto original, não usada pela aplicação atual.

Coluna	Tipo	Chave	Descrição
`num_comanda`	int(11)	PK	Identificador do cadastro
`nome_de_cadastro`	varchar(45)		Nome do cliente
`CPF_de_cadastro`	int(11)		CPF do cliente

4. Regras de negócio e funcionamento

4.1 Como um pedido é gravado

Quando o cliente confirma o pagamento, o arquivo `api/pedido.php` grava os dados nesta ordem, dentro de uma **única transação**:

1. Insere a **comanda**, com o nome e o CPF ou CNPJ, e obtém o `ID_comanda`.
2. Insere o **pedido**, com o `ID_comanda`, a data, o status `Em preparo` e o total, e obtém o `ID_pedido`.

3. Insere um registro em **item_pedido** para cada produto do carrinho.
4. Insere o **pagamento**, com a forma de pagamento, o valor e o tipo de retirada.

Se qualquer uma dessas etapas falhar, a transação é desfeita (`ROLLBACK`) e nada fica gravado pela metade.

4.2 Regras aplicadas

- Os **preços são lidos da tabela `produto`** pelo servidor no momento do pagamento. O valor enviado pelo navegador não é usado, o que evita preços adulterados.
- O total do pedido é a soma de `valor_item_pedido` de todos os itens.
- O CPF (11 dígitos) ou CNPJ (14 dígitos) é validado pelos dígitos verificadores, no navegador e de novo no servidor, e gravado **somente com números**.
- A forma de pagamento e o tipo de retirada só aceitam os valores listados na seção 3.5.
- Cada item do carrinho aceita de 1 a 99 unidades.
- O nome do cliente é cortado em 45 caracteres, o tamanho da coluna.

4.3 Ciclo de vida do status do pedido

A aplicação grava o status `Em preparo` quando o pedido é criado. A coluna aceita até 20 caracteres e permite que uma tela de cozinha ou de retirada atualize o valor depois, por exemplo para `Pronto` ou `Entregue` .

5. Consultas úteis

Listar o cardápio:

```
SELECT ID_produto, nome_produto, preco_produto
FROM produto
ORDER BY nome_produto;
```

Listar os pedidos com o nome do cliente e a forma de pagamento:

```
SELECT p.ID_pedido, c.nome_comanda, p.valorTotal_pedido,
       pg.formaPagamento, pg.retiradaAlimento, p.status_pedido
FROM pedido p
JOIN comanda c    ON c.ID_comanda = p.ID_comanda
JOIN pagamento pg ON pg.ID_pedido = p.ID_pedido
ORDER BY p.ID_pedido DESC;
```

Ver os itens de um pedido (exemplo: pedido 1):

```
SELECT pr.nome_produto, i.quantidade_item_pedido, i.valor_item_pedido
FROM item_pedido i
JOIN produto pr ON pr.ID_produto = i.ID_produto
WHERE i.ID_pedido = 1;
```

Total vendido por forma de pagamento:

```
SELECT formaPagamento, COUNT(*) AS pedidos, SUM(valorTotal_pagamento) AS
total
FROM pagamento
GROUP BY formaPagamento;
```

Produtos mais vendidos:

```
SELECT pr.nome_produto, SUM(i.quantidade_item_pedido) AS vendidos
FROM item_pedido i
JOIN produto pr ON pr.ID_produto = i.ID_produto
GROUP BY pr.ID_produto, pr.nome_produto
ORDER BY vendidos DESC;
```

Atualizar o status de um pedido:

```
UPDATE pedido SET status_pedido = 'Pronto' WHERE ID_pedido = 1;
```

6. Observações e limitações do modelo

- **Tabela `cadastro` sem uso.** O campo `CPF_de_cadastro` é `int(11)`, que não comporta um CPF (11 dígitos) nem um CNPJ. Por isso a aplicação grava o cliente na tabela `comanda`, cujo campo `CPF_comanda` é `varchar(14)`.
- **Sem chave estrangeira em `item_pedido.ID_pedido`.** O dump cria só o índice `fk_pedido`, sem a restrição. Para ligar de fato os itens ao pedido, use o comando comentado no final do `produtos.sql`:

```
ALTER TABLE `item_pedido`
  ADD CONSTRAINT `fk_item_pedido_pedido`
  FOREIGN KEY (`ID_pedido`) REFERENCES `pedido` (`ID_pedido`);
```

- **Data como texto.** `data_pedido` é `varchar(14)` e guarda `AAAAMMDDHHMMSS`. Esse formato permite ordenar corretamente, mas não permite usar as funções de data do MySQL diretamente. Uma evolução seria trocar o tipo por `datetime`.
- **CPF e CNPJ sem máscara.** A coluna `CPF_comanda` guarda apenas números porque um CNPJ formatado (18 caracteres) não caberia em `varchar(14)`. Para saber se é CPF ou CNPJ, basta contar os dígitos (11 ou 14).

- **Sem categorias de produto.** A tabela `produto` não tem categoria (lanches, bebidas, sobremesas). O cardápio aparece numa lista única.
- **Pagamento simulado.** A tabela `pagamento` só registra a forma escolhida e o valor. Não há integração com operadoras de cartão ou PIX.

7. Instalação e manutenção do banco

7.1 Criar o banco

1. Ligue o **Apache** e o **MySQL** no XAMPP.
2. Abra `http://localhost/phpmyadmin` e crie um banco chamado `alimentacao`, com ordenação `utf8mb4_general_ci`.
3. Selecione o banco, abra a aba **Importar** e envie o `alimentacao.sql`. Isso cria as 6 tabelas.
4. Na mesma aba, envie o `produtos.sql`, **uma única vez**, para cadastrar os 10 lanches do cardápio.

7.2 Conexão da aplicação

Os parâmetros de conexão ficam no arquivo `api/conexao.php`:

Parâmetro	Valor padrão
Servidor	`localhost`
Banco	`alimentacao`
Usuário	`root`
Senha	(vazia)
Charset	`utf8mb4`

7.3 Cuidados

- Importe o `alimentacao.sql` **antes** do `produtos.sql`. Sem as tabelas, o erro é `#1146 – Tabela 'alimentacao.produto' não existe`.
- Não apague um produto que já tenha sido vendido, porque `item_pedido` aponta para ele. Para tirar um lanche da venda, o caminho mais seguro é criar uma coluna de disponibilidade. O banco atual não tem essa coluna.
- Para voltar ao zero, apague as linhas de `pagamento`, `item_pedido`, `pedido` e `comanda`, nessa ordem, ou recrie o banco e reimporte os dois arquivos.

- Para fazer backup, use a aba **Exportar** do phpMyAdmin, com o banco `alimentacao` selecionado.